Vector DB & RAG 검색 성능 비교 실습

1. 프로젝트 개요

본 실습에서는 AI·반도체 관련 PDF 문서를 대상으로 Vector DB 기반 검색 환경을 구축하고, 검색 방식을 단계적으로 확장하면서 검색 품질과 RAG 성능이 어떻게 달라지는지 비교하였다.

실습은 크게 두 단계로 구성하였다.

- **실습 1**: PDF Loading → Chunking → BGE-M3 Embedding → FAISS → Qdrant → Metadata Filter
- **실습 2**: Dense Search → BM25 → RRF 기반 Hybrid Search → Reranker → RAGAS 평가

최종 비교 질의는 다음과 같이 통일하였다.

```
HBM이란 무엇인가?
```

---

2. 프로젝트 구조

```
vector_db/
├── README.md
├── requirements.txt
├── .env.example
├── src/
│   ├── 01_load_chunk.py
│   ├── 02_embedding.py
│   ├── 03_query_embedding.py
│   ├── 04_faiss_search.py
│   ├── 05_qdrant_upsert.py
│   ├── 06_qdrant_search.py
│   ├── 07_query_hbm_embedding.py
│   ├── 08_dense_bm25_compare.py
│   ├── 09_rrf.py
│   ├── 10_reranker.py
│   ├── 11_compare_three_stages.py
│   ├── 12_ragas_eval.py
│   └── bm25_utils.py
├── results/
│   ├── practice1_faiss.txt
│   ├── practice1_qdrant_filter.txt
│   ├── practice2_dense_vs_bm25_final.txt
│   ├── practice2_rrf_final.txt
│   ├── practice2_reranker_final.txt
│   ├── practice2_three_stage_final.txt
│   ├── practice2_ragas_final.txt
│   ├── three_stage_comparison.csv
│   ├── ragas_results.csv
│   └── ragas_details.json
```

```
├── data/
└── artifacts/
```

> `.env`, `.venv`, `qdrant_storage`, 캐시 및 모델 캐시는 제출 대상에서 제외한다.

---

3. 실습 환경 및 데이터

총 16개의 AI·반도체 관련 PDF 문서를 대상으로 실습을 진행하였다.

항목	결과
PDF 문서 수	16
추출 페이지 수	843
생성 Chunk 수	2,399
Chunk Size	500
Chunk Overlap	50
Embedding Model	`BAAI/bge-m3`
Embedding Dimension	1,024
Embedding Shape	`(2399, 1024)`
전체 임베딩 시간	약 26.82초

문서에서 텍스트를 추출한 뒤 `RecursiveCharacterTextSplitter`를 이용해 Chunk를 생성하고, BGE-M3로 1,024차원 임베딩을 생성하였다.

---

4. 실습 1 — Vector DB 구축 및 검색

4.1 PDF Loading & Chunking

PDF 문서를 페이지 단위로 읽고 다음 Metadata를 함께 저장하였다.

- `source`: 원본 PDF 파일명
- `page`: PDF 페이지 번호
- `text`: Chunk 본문

이후 `chunk_size=500`, `chunk_overlap=50`으로 총 2,399개의 Chunk를 생성하였다.

4.2 BGE-M3 Embedding

생성된 Chunk 전체에 대해 `BAAI/bge-m3` 모델로 임베딩을 생성하였다.

```
Embedding Shape: (2399, 1024)
```

```
dtype: float32
```

생성된 임베딩은 이후 FAISS와 Qdrant 검색에 동일하게 활용하였다.

4.3 FAISS Search

FAISS HNSW 인덱스를 구축하고 다음 질의를 검색하였다.

```
HBM이란 무엇인가?
```

검색 결과 HBM의 구조, TSV 기반 적층 방식, 메모리 병목, AI 반도체에서의 활용 등을 설명하는 문서가 상위에 검색되었다.

주요 검색 내용은 다음과 같았다.

- HBM의 DRAM 적층 및 TSV 구조
- HBM의 고대역폭 특성
- AI 연산에서의 메모리 병목
- AI GPU/NPU에서 HBM의 중요성

상세 실행 결과:

- `results/practice1_faiss.txt`

4.4 Qdrant Search & Metadata Filter

동일한 임베딩을 Qdrant의 `rag_docs` Collection에 저장하였다.

각 Point에는 Vector와 함께 다음 Payload를 저장하였다.

```
text
source
page
```

일반 Vector Search 외에도 다음 Metadata Filter를 적용하였다.

```
page >= 5
```

필터 적용 후 반환된 결과가 모두 조건을 만족하는 것을 확인하였다.

FAISS와 Qdrant 모두 의미 기반 검색이 가능했지만, Qdrant에서는 문서 출처, 페이지 등 Metadata 조건을 함께 활용할 수 있었다.

실제 RAG 서비스에서는 작성일, 문서 유형, 부서, 접근 권한 등으로 검색 범위를 제한해야 할 수 있으므로 Metadata 기반 Filtering이 중요하다고 판단하였다.

상세 실행 결과:

- `results/practice1_qdrant_filter.txt`

---

5. 실습 2 — Dense → Hybrid(RRF) → Reranker

실습 2에서는 동일한 질문에 대해 검색 방식을 단계적으로 추가하며 결과 변화를 비교하였다.

```
HBM이란  무엇인가?
```

5.1 Dense Search

BGE-M3 임베딩 기반 Dense Search에서는 HBM과 의미적으로 관련된 문서가 안정적으로 검색되었다.

Dense Top-5에는 다음 내용이 포함되었다.

- HBM 구조 및 TSV 적층 방식
- HBM과 메모리 병목
- HBM의 대용량·고대역폭 특성
- HBM 산업 및 시장 관련 내용
- HBM과 AI 가속기 경쟁력

Dense Search는 단순 문자열 일치가 아니라 질의와 문서의 의미적 유사성을 기반으로 검색한다는 점을 확인할 수 있었다.

5.2 BM25 Search

Dense Search와 별도로 BM25 기반 Sparse Search를 구성하였다.

BM25는 질의에 포함된 키워드와 문서 내 토큰의 직접적인 일치를 이용하기 때문에 제품명, 약어, 코드와 같이 정확한 문자열이 중요한 검색을 보완할 수 있다.

BM25 Troubleshooting

초기에는 기본 Tokenizer가 다음과 같이 질의를 처리하였다.

```
HBM이란  무엇인가?
→ ['hbm이란', '무엇인가']
```

문서에는 `HBM`, `HBM은`, `HBM의`와 같은 형태가 주로 존재했기 때문에 `HBM이란`이 별도 토큰으로 처리되면서 BM25 Top-5의 점수가 모두 `0.0000`으로 나타났다.

이를 해결하기 위해 영문·숫자와 한글을 분리하는 간단한 Custom Splitter를 적용하였다.

```
HBM이란  무엇인가?
→ ['hbm', '이란', '무엇인가']
```

수정 후에는 HBM 토큰이 문서의 HBM과 직접 매칭되면서 BM25가 정상적으로 점수를 계산하였다.

순위	주요 내용	BM25 Score
1	HBM·메모리 병목	2.7722
2	HBM 시장·기술	2.7466
3	ETRI 비디오 코딩	2.7283
4	ETRI AI 통신 기술	2.6857
5	HBM 공급망	2.5593

BM25가 정상 동작한 이후에도 일부 관련성이 낮은 문서가 상위에 포함되었다. 이는 Sparse Search가 문서의 의미 전체보다 토큰의 직접 매칭에 영향을 받기 때문이다.

이번 실습에서는 특정 질의에 맞춘 추가 Stopword 튜닝까지 진행하기보다, 이러한 후보를 Hybrid Search와 Reranker 단계에서 어떻게 보완할 수 있는지 확인하는 방향으로 진행하였다.

상세 실행 결과:

- `results/practice2_dense_vs_bm25_final.txt`

5.3 Hybrid Search — Dense + BM25 + RRF

Dense Search와 BM25의 순위를 RRF(Reciprocal Rank Fusion)로 결합하였다.

최종 Hybrid Top-5는 다음과 같았다.

순위	ID	Dense Rank	BM25 Rank	주요 내용
1	2365	2	1	HBM과 메모리 병목
2	2368	4	2	HBM 시장·기술
3	1926	1	7	HBM 구조 및 TSV
4	2366	3	10	HBM 정의 및 고대역폭 특성
5	2065	7	8	HBM과 AI 성능 병목

Dense와 BM25 양쪽에서 높은 순위를 기록한 HBM 관련 문서가 RRF 상위에 배치되었다.

반면 BM25에서만 높은 순위를 기록했던 ETRI 비디오 코딩, AI 통신 관련 문서는 RRF 9~10위로 밀렸다.

즉 이번 실험에서는 **RRF가 Dense와 BM25의 서로 다른 검색 결과를 결합하면서 한쪽 검색기의 Noise가 최종 상위권에 미치는 영향을 줄이는 효과**를 확인할 수 있었다.

상세 실행 결과:

- `results/practice2_rrf_final.txt`

5.4 Reranker

RRF를 통해 생성한 후보 문서 10개를 `BAAI/bge-reranker-v2-m3`로 다시 평가하였다.

최종 Top-5는 다음과 같았다.

순위	ID	RRF Rank	Reranker Score	주요 내용
1	2023	8	0.9823	HBM의 역할과 AI 반도체 데이터 공급
2	2365	1	0.9258	HBM과 메모리 병목
3	1926	3	0.8975	HBM 구조 및 TSV
4	2366	4	0.8805	HBM 정의 및 고대역폭 특성
5	2065	5	0.8544	HBM과 AI 성능 병목

특히 RRF 8위였던 문서가 Reranker 1위로 상승하였다. 해당 문서는 HBM을 AI 반도체에 필요한 데이터를 고속으로 공급하는 메모리로 직접 설명하고 있어, `"HBM이란 무엇인가?"`라는 질문에 대한 직접적인 관련성이 높았다.

반대로 HBM 시장 및 공급 부족과 같이 질문의 정의적 의도와 거리가 있는 문서는 상대적으로 순위가 내려갔다.

이를 통해 **RRF가 후보군을 결합하고, Reranker가 질문과 후보 문서를 다시 직접 비교해 최종 순위를 정밀하게 조정하는 구조**를 확인할 수 있었다.

상세 실행 결과:

- `results/practice2_reranker_final.txt`

5.5 Dense → Hybrid → Reranker 비교

최종 Top-5를 비교하면 다음과 같다.

Rank	Dense	Hybrid(RRF)	Reranker
1	HBM 구조/시장	HBM·메모리 병목	HBM의 역할·데이터 공급
2	HBM·메모리 병목	HBM 시장·기술	HBM·메모리 병목
3	HBM 정의/특징	HBM 구조/TSV	HBM 구조/TSV
4	HBM 시장/기술	HBM 정의/특징	HBM 정의/특징
5	HBM·AI 가속기	HBM·AI 성능 병목	HBM·AI 성능 병목

Dense Search 자체도 높은 품질의 결과를 보여주었다.

Hybrid Search에서는 Dense와 BM25의 서로 다른 후보가 결합되었고, 양쪽에서 공통으로 높은 평가를 받은 HBM 관련 문서가 상위에 배치되었다.

Reranker 단계에서는 질문의 의도와 후보 문서를 직접 비교하면서 **HBM의 정의와 기술적 역할을 더 직접적으로 설명하는 문서가 상위로 재배치**되었다.

따라서 이번 실험에서는 다음과 같은 구조를 확인하였다.

```
Dense + BM25
      ↓
RRF로 후보 결합
```

```
      ↓
다양한 HBM 관련 후보 확보
      ↓
Reranker로 질문–문서 재평가
      ↓
질문에 직접 답하는 Context를 상위 배치
```

상세 비교 결과:

- `results/practice2_three_stage_final.txt`
- `results/three_stage_comparison.csv`

---

6. RAGAS 평가

각 검색 방식의 Top-K Context를 기반으로 답변을 생성한 뒤 RAGAS로 정량 평가하였다.

사용한 지표는 다음과 같다.

- **Faithfulness**: 생성 답변이 검색 문서의 내용에 근거하는가
- **Answer Relevancy**: 생성 답변이 사용자 질문의 의도에 적절하게 답하는가
- **Context Precision**: 검색된 Context 중 질문에 실제로 유용한 문서가 얼마나 적절하게 상위에 위치하는가

6.1 최종 평가 결과

검색 방식	Faithfulness	Answer Relevancy	Context Precision
Dense	**1.0000**	0.7322	0.9500
Hybrid (RRF)	**1.0000**	0.7091	0.8042
Reranker	**1.0000**	**0.7428**	**1.0000**

상세 결과:

- `results/practice2_ragas_final.txt`
- `results/ragas_results.csv`
- `results/ragas_details.json`

6.2 결과 분석

Faithfulness

세 방식 모두 `1.0000`으로 나타났다.

따라서 Dense, Hybrid, Reranker 기반으로 생성한 답변 모두 검색 문서의 내용을 벗어나지 않고 근거에 충실하게 생성되었다고 볼 수 있다.

Answer Relevancy

Answer Relevancy는 다음과 같이 나타났다.

```
Dense       0.7322
Hybrid      0.7091
Reranker    0.7428
```

Reranker가 가장 높은 점수를 기록하였다.

이는 Reranker가 `"HBM이란 무엇인가?"`라는 질문에 직접 답하는 문서를 상위에 배치하면서 최종 생성 답변 역시 질문 의도에 더 잘 맞게 구성된 결과로 해석할 수 있다.

다만 하나의 질의에 대한 평가 결과이므로 이 점수만으로 특정 검색 방식이 항상 우수하다고 일반화하기는 어렵다.

Context Precision

Context Precision은 다음과 같이 나타났다.

```
Dense       0.9500
Hybrid      0.8042
Reranker    1.0000
```

Dense Search는 처음부터 높은 품질의 관련 문서를 검색하였다.

Hybrid Search에서는 Dense와 BM25의 다양한 후보를 결합하면서 Context의 관련성에 차이가 생겨 점수가 `0.8042`로 낮아졌다.

이후 Reranker가 질문과 각 후보의 관련성을 다시 평가하여 HBM의 정의와 기술적 역할을 직접 설명하는 Context를 상위에 배치하면서 Context Precision이 `1.0000`으로 향상되었다.

즉 이번 실험에서는 **Hybrid가 후보를 결합하고 Reranker가 최종 정밀도를 보완하는 구조가 RAGAS 결과에서도 확인되었다.**

---

7. RAG 설정 시 주요 고려사항

이번 실습을 통해 RAG 성능은 특정 Vector DB나 Embedding Model 하나로 결정되는 것이 아니라, **Chunking → Retrieval → Filtering → Reranking → Evaluation의 전체 파이프라인 설정에 영향을 받는다**고 판단하였다.

7.1 Chunking

Chunk가 너무 작으면 검색 위치는 정밀해질 수 있지만 필요한 문맥이 여러 Chunk로 나뉠 수 있다.

반대로 너무 큰 Chunk는 하나의 Chunk 안에 여러 주제가 섞여 검색 정확도를 낮출 수 있다.

따라서 문서 구조와 실제 사용자 질의 특성을 기준으로 Chunk Size와 Overlap을 조정해야 하며, 긴 문맥이 필요한 경우 Parent-Child 방식도 고려할 수 있다.

7.2 Dense와 Sparse Search

Dense Search는 표현이 다르더라도 의미적으로 유사한 문서를 찾는 데 강점이 있다.

반면 BM25는 제품명, 코드, 약어처럼 정확한 키워드 매칭이 중요한 검색을 보완할 수 있다.

이번 실습에서는 BM25 Tokenization 문제로 검색 점수가 모두 0이 되는 상황을 확인했고, 간단한 Custom Splitter를 적용해 이를 해결하였다.

이를 통해 **Sparse Search에서는 단순히 BM25 알고리즘을 사용하는 것뿐 아니라 언어와 데이터 특성에 맞는 Tokenization도 검색 품질에 직접적인 영향을 준다**는 점을 확인하였다.

7.3 Hybrid Search

Hybrid Search는 Dense와 Sparse Search의 장점을 함께 활용할 수 있지만, 두 Retriever 중 하나의 품질이 낮으면 잘못된 후보도 함께 결합될 수 있다.

따라서 Hybrid Search를 적용하기 전에 각 Retriever가 독립적으로 정상 동작하는지 확인하는 과정이 필요하다고 생각한다.

이번 실습에서도 BM25 Tokenization을 수정한 이후 RRF Top-5가 HBM 관련 문서로 구성되면서 Hybrid 결과가 안정적으로 개선되었다.

7.4 Metadata Filter

실제 서비스에서는 전체 문서 중 의미가 비슷한 문서를 검색하는 것만으로 충분하지 않을 수 있다.

예를 들어 다음과 같은 조건이 필요할 수 있다.

- 최신 문서만 검색
- 특정 부서의 문서만 검색
- 특정 문서 유형만 검색
- 사용자에게 접근 권한이 있는 문서만 검색

따라서 Vector Search와 Metadata Filter를 함께 설계하는 것이 중요하다.

7.5 Reranker

Reranker는 질문과 후보 문서를 직접 비교하기 때문에 최종 검색 정밀도를 높이는 데 효과적이었다.

이번 실험에서도 RRF 8위였던 문서가 질문의 의도와 높은 관련성을 인정받아 Reranker 1위로 상승하였다.

다만 추가 모델 추론이 필요하므로 검색 후보 수가 많아질수록 응답 시간과 연산 비용이 증가한다.

따라서 모든 문서를 Reranker에 입력하기보다 1차 검색에서 Top-N 후보를 추린 후 제한된 후보만 재평가하는 방식이 적절하다고 생각한다.

7.6 Evaluation

한두 개의 예시 질문만으로 RAG 구조의 우열을 판단하기는 어렵다.

이번 실험에서도 Dense → Hybrid → Reranker 순으로 검색 구조를 고도화했지만 모든 RAGAS 지표가 순차적으로 증가한 것은 아니었다.

따라서 실제 서비스에서는 다음과 같이 성격이 다른 평가 질의를 구성할 필요가 있다.

- 단일 문서에서 바로 답할 수 있는 질문
- 정확한 키워드가 중요한 질문
- 여러 문서를 종합해야 하는 질문
- Metadata Filter가 필요한 질문

또한 RAGAS와 같은 정량 평가뿐 아니라 실제 검색 결과를 직접 확인하는 정성 평가도 함께 수행해야 한다.

결국 좋은 RAG 시스템은 가장 많은 검색 기술을 적용한 시스템이 아니라, **실제 데이터와 질의 특성에 맞춰 검색 품질·응답 속도·비용·운영 복잡도의 균형을 검증한 시스템**이라고 생각한다.

---

8. 결론

이번 실습에서는 16개의 PDF 문서를 2,399개의 Chunk로 구성하고 BGE-M3 기반 Vector Search 환경을 구축하였다.

FAISS와 Qdrant를 이용하여 Vector Search를 수행하였고, Qdrant에서는 Metadata Filter를 적용하여 Vector DB의 활용 방식을 확인하였다.

이후 Dense Search에 BM25와 RRF를 결합하고 BGE Reranker를 적용하여 검색 결과 변화를 비교하였다.

실습 과정에서 BM25의 기본 Tokenization으로 인해 `HBM이란 무엇인가?` 질의가 문서의 `HBM`과 직접 매칭되지 않아 모든 BM25 점수가 0으로 나타나는 문제를 확인하였다. 이를 Custom Splitter로 수정하여 BM25를 정상화한 뒤 Hybrid Search를 다시 수행하였다.

최종적으로 RRF는 Dense와 BM25의 후보를 효과적으로 결합하였고, Reranker는 질문에 직접 답하는 문서를 상위에 재배치하였다.

RAGAS 평가에서는 세 방식 모두 Faithfulness가 `1.0000`이었으며, Reranker가 Answer Relevancy `0.7428`, Context Precision `1.0000`으로 가장 높은 최종 결과를 기록하였다.

이를 통해 **검색 기술을 단순히 추가하는 것보다 각 Retriever가 정상적으로 동작하는지 확인하고, 실제 검색 결과와 평가 지표를 기반으로 단계별 효과를 검증하는 것이 더 중요하다**는 점을 확인하였다.

---

9. 실행 순서

가상환경 활성화 후 필요한 패키지를 설치한다.

```
pip install -r requirements.txt
```

환경 변수는 `.env.example`을 참고하여 `.env`에 설정한다.

```
OPENAI_API_KEY=your_openai_api_key_here
RAGAS_MODEL=gpt-6-luna
```

주요 실습 실행 순서는 다음과 같다.

```
python src/01_load_chunk.py
python src/02_embedding.py
python src/03_query_embedding.py
python src/04_faiss_search.py
python src/05_qdrant_upsert.py
python src/06_qdrant_search.py

python src/07_query_hbm_embedding.py
python src/08_dense_bm25_compare.py
python src/09_rrf.py
python src/10_reranker.py
python src/11_compare_three_stages.py

RAGAS_MODEL=gpt-6-luna python src/12_ragas_eval.py
```

> Qdrant 실행 환경이 필요한 경우 Qdrant 서버를 먼저 실행한 뒤 `05_qdrant_upsert.py`, `06_qdrant_search.py`를 실행한다.